JustGoal: With Only a Goal, the AI Baseline Is Above the Expert Kernels

Yaojian Chen[1*], Dazhi Peng[2*], Zhui Zhu[1], Leshan Li[1], Haojun Sun[2], Yiyun Zheng[2], Qingfeng He[1], Shangzhan Li[3], Xianglong Deng[4], Yifei Shen[2], Shengye Xiang[1], Zhen Li[2], Lin Gan[1], Yuxuan Li[2†], Xu Han[1†], Zhiyuan Liu[1†]

[1]Tsinghua University [2]ModelBest Inc. [3]Harbin Institute of Technology
[4]University of Chinese Academy of Sciences

Abstract

A small number of GPU kernels often determines the efficiency of large language model training and inference. High-performance kernel development still typically draws on expert knowledge, from direct expert implementation, to collaboration with general-purpose models, to specialized agents that encode expert-designed evaluation methods, domain knowledge, and search workflows. These approaches differ mainly in how expert knowledge enters the development process. We ask how much additional value that knowledge still provides for kernel optimization as foundation models grow stronger. To study this question, we construct a goal-only baseline and call its concrete experimental configuration JustGoal: a general-purpose agent receives only the optimization objective and constraints, with no kernel-specific evaluation infrastructure, knowledge base, or search workflow, and builds its own evaluation loop. On NVIDIA B200 GPUs, JustGoal outperforms open-source libraries on 8 of the 10 operators entering the comparison, with per-operator geometric mean speedups of 1.14–2.34×, and surpasses the fastest publicly released specialized-agent kernel on 5 of the 9 operators that have one. This marks a capability threshold: without task-time injection of kernel-specific knowledge or optimization workflows, current frontier models can already surpass human-maintained implementations on a range of operators. In a controlled same-model comparison with Fable 5.1 on single shapes, the KDA-0.5 harness reaches 0.95× the performance of JustGoal, and single-mechanism ablations find no consistent benefit from domain knowledge or predefined workflows. These results show that the effectiveness of specialized machinery cannot be presumed from its complexity or from results on earlier models. It should be established by a clear gain over a goal-only baseline under the same model and task scope, and reevaluated when the foundation model changes.

1 Introduction

The computational efficiency of large language model training and inference depends critically on a small set of GPU kernels. Developing high-performance implementations of these kernels requires jointly optimizing tiling strategies, memory hierarchy utilization, asynchronous execution pipelines, and hardware-specific instructions. These design choices are tightly coupled, and their optimal configuration changes across hardware generations. Consequently, the development of high-performance kernels has long relied on leading experts devoting weeks or even months to manual optimization. The FlashAttention series exemplifies this recurring investment: achieving high performance on successive GPU architectures has required substantial redesigns of its algorithms and execution pipelines (Dao et al., 2022; Shah et al., 2024; Dao et al., 2026).

[*]Equal contribution.
[†]Corresponding authors: Yuxuan Li, Xu Han, Zhiyuan Liu.

Large language models have recently entered kernel development. Experts may collaborate directly with general-purpose models. They may also encode evaluation methods, domain knowledge, and search workflows into specialized kernel agents such as KDA, AVO, AKO, and CAKE. These agents have matched or surpassed expert-written implementations on selected operators (MIT HAN Lab, 2026; NVIDIA, 2026; AKO Team, 2026a; Ye et al., 2026). What these approaches share is their reliance on expert knowledge. As foundation models improve, capabilities that previously had to be supplied explicitly may become native to the model. This raises two related questions: how much additional value does expert knowledge continue to provide for kernel optimization as model capabilities increase, and how far can a model optimize kernels when given only the task objective and constraints, with as little predefined expert knowledge as possible?

To examine these questions, we construct a goal-only baseline and call its concrete experimental configuration JustGoal. It provides no kernel-specific evaluation infrastructure, domain knowledge, or search workflow. A general-purpose agent receives only the task objective and constraints. It builds its own evaluation loop and optimization process. We observe the role of expert knowledge from two measurable perspectives. First, we compare kernels generated by JustGoal with human-maintained open-source libraries and publicly released specialized-agent kernels, asking what performance level a goal-only configuration already reaches. Second, under the same model and task scope, we compare JustGoal with a specialized harness that adds evaluation infrastructure, domain knowledge, and search workflows, directly measuring the marginal value of these explicit expert mechanisms. Our experiments cover 11 operators spanning attention, linear attention, sparse attention, and mixture-of-experts (MoE). We independently verify every kernel in the comparison.

From the first perspective, JustGoal is faster than human-maintained open-source libraries on 8 of the 10 operators entering the comparison, and faster than the best publicly released specialized-agent kernel on 5 of the 9 operators that have one. This marks a capability threshold: without task-time injection of kernel-specific knowledge or optimization workflows, current frontier models can construct evaluation feedback, identify performance bottlenecks, and surpass human-maintained implementations on a range of operators. From the second perspective, in the scope-matched Fable 5.1 comparison, the full KDA-0.5 harness produces no overall gain on single-shape tasks, and single-mechanism ablations show no consistent gain from domain knowledge or predefined workflows. When the goal expands from optimizing one shape to covering every workload in an operator definition, the harness still provides a gain. To measure the net gain that the full specialized harness adds beyond the model's own capabilities, it must be compared with a goal-only baseline under the same model and task scope. This net gain should be reevaluated whenever the foundation model changes.

2 Related Work

2.1 High-Performance Kernel Design

Hardware–algorithm co-design. High-performance kernel design requires aligning an algorithm's computational and data movement demands with the resources of the target hardware. The FlashAttention series exemplifies this hardware–algorithm co-design: FlashAttention uses tiling to reduce data movement between high-bandwidth memory (HBM) and on-chip SRAM (Dao et al., 2022); FlashAttention-3 reorganizes computation and data transfers around Hopper's asynchronous execution capabilities (Shah et al., 2024); and FlashAttention-4 further adapts the algorithm and execution pipeline to changes in the balance of hardware resources, addressing bottlenecks such as exponential evaluation and shared-memory access (Dao et al., 2026). This progression illustrates that, even when an operator's mathematical definition remains unchanged, achieving high performance requires experts to reassess bottlenecks and adapt implementations to the target hardware.

New operators. Beyond hardware advances, evolving model architectures continually introduce new demands for kernel development and optimization. Existing kernel implementations may not directly meet the performance requirements of new operators, necessi-

tating targeted development and optimization. For example, DeepSeek provides FlashMLA for multi-head latent attention (MLA) (DeepSeek-AI, 2025b). With the introduction of DeepSeek Sparse Attention (DSA), it also provides the corresponding sparse attention kernels through FlashMLA and implements the indexer's scoring kernels in DeepGEMM (DeepSeek-AI, 2025c;a). These efforts illustrate that high-performance kernel development requires sustained expert design and optimization not only to adapt to hardware changes but also to keep pace with evolving model architectures.

2.2 LLM-Based Kernel Generation

Execution feedback and search. KernelBench evaluates models' ability to generate faster, semantically equivalent kernels from PyTorch reference implementations and explores iterative refinement using execution and profiling feedback (Ouyang et al., 2025). Existing work uses different search mechanisms to organize this optimization: METR uses parallel tree search (METR, 2025), AI CUDA Engineer uses evolutionary search (Sakana AI, 2025), K-Search guides search with a co-evolving world model (Cao et al., 2026), and AdaExplore distills recurring failures into reusable validity rules (Du et al., 2026). AVO introduces autonomous agents into the mutation step of evolutionary search, producing attention kernels that outperform expert implementations on some evaluated configurations (NVIDIA, 2026).

Evaluation reliability. Optimization can exploit flaws in evaluation protocols. KernelBench-Verified reports that a GPT-5.5-generated ReLU kernel simply returns its input for known test shapes, yielding an apparent 374× speedup (Zhang et al., 2026). To guard against such failures, kernel agents commonly incorporate independent validation, including sanitizer and anti-cheating checks in AKO4X (AKO Team, 2026c) and independent acceptance checks by Codex in KDA (HAN Lab Kernel Mafia, 2026).

Domain knowledge and development environments. Timing reveals that a kernel is slow, but not why. Another line of work therefore equips agents with domain knowledge and diagnostic tools. CudaForge uses a Judge agent to interpret NCU metrics and guide revisions by a Coder agent (Zhang et al., 2025); KDA identifies the interface between models and the engineering loop as a primary bottleneck, motivating KernelWiki, built from production-code pull requests, an NCU profiling skill, and an IKET (in-kernel event tracing) profiling skill (MIT HAN Lab, 2026; HAN Lab Kernel Mafia, 2026); AKO4X maintains DSL-specific skills and a library of operator pitfalls (AKO Team, 2026c); and CAKE lets agents directly edit typed IR, with the compiler providing localized diagnostics before compilation (Ye et al., 2026).

General-purpose agent foundations. General-purpose agents such as Claude Code and Codex provide domain-independent tool use, context management, and long-running autonomous execution (Anthropic, 2025; OpenAI, 2025). Kernel agents typically build on these foundations—KDA and AKO use Claude Code, while AVO uses an internal general-purpose agent—and add kernel-specific mechanisms for evaluation, knowledge, and workflow (MIT HAN Lab, 2026; AKO Team, 2026a; NVIDIA, 2026). Table 1 summarizes these mechanisms for the systems compared in Section 4.

These approaches share a common pattern: each agent limitation motivates an additional mechanism. Yet these mechanisms were designed and validated with the underlying models available at the time; as those models improve, the marginal value of each mechanism must be measured again. JustGoal removes these kernel-specific layers and instantiates the goal-only baseline used for that measurement.

3 JustGoal: With Only a Goal

3.1 JustGoal

The box in Figure 1 contains the complete input received by the agent. Here, "goal-only baseline" denotes a control principle: give a general-purpose agent only the objective and

Table 1: Kernel-specific mechanisms added to general-purpose agent foundations. KDA and AKO entries are based on their repositories and technical reports (MIT HAN Lab, 2026; HAN Lab Kernel Mafia, 2026; AKO Team, 2026b;c); AVO and CAKE entries are based on their papers (NVIDIA, 2026; Ye et al., 2026).

System	Evaluation	Knowledge	Workflow
KDA	KDA-0.1 uses the FlashInfer-Bench standard; KDA-0.5 not released	KernelWiki (production PR corpus); NCU profiling skill; IKET (in-kernel event tracing) profiling skill	humanize2-based workflow; three-stage research–iterate–tune prompts; independent Codex acceptance checks at each step
AVO	Scoring function: correctness checks against a reference and throughput measurement	Knowledge base: CUDA/PTX documentation, Blackwell specifications, and FA4 source code	Single-lineage evolution; only non-regressing versions are committed; supervisor intervention upon stagnation
AKO	Compilation, correctness (including sanitizer), and timing benchmarks; anti-cheating checks	DSL-specific skills (including NCU profiling) and an operator-pitfall library (AKO4X)	Profile–modify–benchmark–commit loop with stagnation rules; multi-round campaigns with main and sub-agents (AKO4X)
CAKE	Pre-compilation validation of typed IR, cost-model ranking, localized diagnostics, and CUPTI timing	Recurring failures distilled into validation rules, IR primitives, and optimization tactics	Generate–filter–evaluate–route stages; single-shape evolution followed by library generalization
JustGoal	—	—	—

This is the target operator we want: {operator config}. First measure its device time on a B200. Starting from scratch, and never looking at or copying the reference implementation, write a kernel that is at least 1.2× faster in device time.

1. Use an isolated, clean environment; do not use existing skills, memory, or other local files.
2. Make sure the kernel is correct. Preferably, first write a slow naive fp32 implementation and compare it with the reference; their difference gives a lower bound on the tolerance, which you multiply by a factor and then use to compare your implementation against the slow fp32 implementation.
3. Do not ask questions; run in `--dangerously-skip-permissions` mode.
4. Use the B200 cluster for the GPU.

Figure 1: The complete task input for JustGoal (English translation; the original Chinese prompt is given in Appendix A).

constraints, without adding kernel-specific machinery. JustGoal is the concrete experimental configuration used in this paper, not a new agent architecture or execution mode. It retains only the general-purpose Claude Code agent substrate and access to a B200 GPU, while removing the entire kernel-specific layer listed in Table 1. Item 1 requires a clean environment precisely to ensure this.

The model builds its own evaluation setup. The agent writes the compilation, correctness-checking, and timing code within its optimization loop: it first measures the reference kernel's device time as the target to surpass, then writes a naive fp32 implementation as its correctness oracle.

We explicitly note that the prompt contains one piece of expert guidance: item 2 instructs the agent to set the tolerance by multiplying the reference implementation's error relative to the naive fp32 implementation by a factor. This is a common numerical-validation practice and does not involve kernel-optimization knowledge.

3.2 Evaluation Protocol

Baseline: open-source library kernels. Candidate implementations are human-maintained libraries only: the official FlashAttention-4 kernels and FlashInfer (including FlashKDA). For each shape, we select the fastest candidate among the implementations included in our evaluation that pass the correctness checks.

Baseline: kernels generated by kernel agents. Candidate implementations are of two kinds: the MLSys'26 contest kernels, and the publicly released kernels from KDA-0.5, AKO4X, CAKE, and AVO (MIT HAN Lab, 2026; AKO Team, 2026c; Ye et al., 2026; NVIDIA, 2026). The contest kernels are likewise produced by kernel agents, so they belong to this category rather than to the open-source libraries. For each shape, we select the fastest candidate among the implementations included in our evaluation that pass the correctness checks. No publicly released specialized-agent kernel for FA4 backward is included in this comparison. Per-shape sources and selection details are provided in Appendix C.

Correctness verification. All kernels included in the comparison are validated with FlashInfer-Bench, except FA4 backward, which is checked following the official FA4 test protocol of Tri Dao. No agent-defined tolerances are used. All final kernels are reviewed by the authors; kernels found numerically wrong are fixed and re-timed, and the text reports the fixed numbers.

Timing methodology. All kernels are timed by the authors in the same environment with FlashInfer-Bench, with the L2 cache flushed before measurement (cold L2). Kernel time is recorded with CUPTI as the kernel span, from the start of the first kernel to the end of the last kernel launched by the entry point, excluding JIT compilation, the cache flush, and synchronization overhead.

4 Results

4.1 Experimental Configuration

The models are Anthropic's Opus 4.8, Opus 5, and Fable 5.1, with reasoning effort `high` throughout; the agent environment is Claude Code 2.1.304. Kernel development and performance measurements run on NVIDIA B200 GPUs with driver 580.105.08 and CUDA 13.0. Baselines and timing follow the protocol of Section 3.2. The operators, shapes, and comparison groups of each experiment are given at the start of its subsection.

4.2 Overview: Comparison with Open-Source Libraries and Kernel Agents

This section gives an overview of JustGoal's results on 11 operators and 42 shapes and answers two different questions: whether JustGoal's kernels are faster than human-maintained open-source libraries, and whether they are faster than the kernels of specialized kernel agents. Each shape is one session, with Fable 5.1 as the primary model; a few sessions were completed by other models after a model fallback. We report geometric mean speedups per operator; per-shape latencies, the models actually used, and baseline sources are in Appendix C.

Against open-source libraries. The blue bars in Figure 2 are JustGoal's results against open-source libraries. Excluding the DSA top-k indexer, whose baseline protocol remains unverified, 10 operators enter the comparison, and JustGoal is faster than the library on 8 of them, with geometric means from 1.14× on FA4 backward to 2.34× on GDN forward; it falls behind on FA4 forward (0.77×) and Kimi Delta Attention (0.99×).

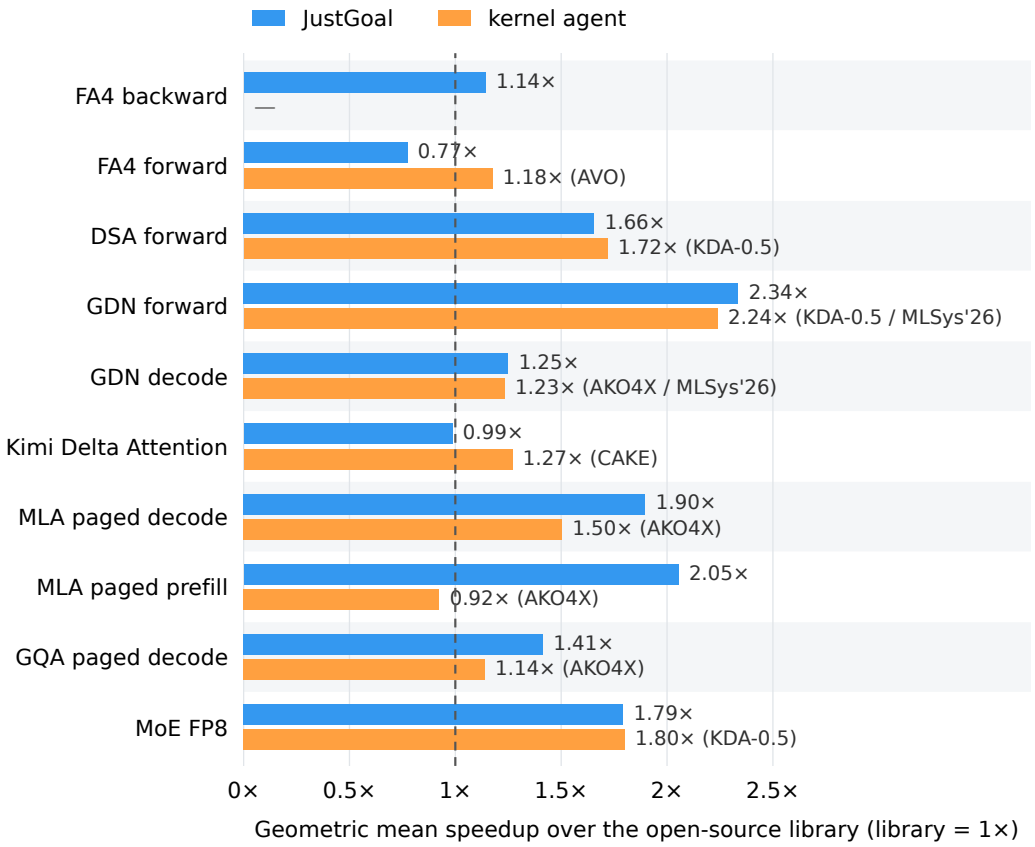


Figure 2: Geometric mean speedups of JustGoal (blue) and of kernel agents (orange) over the open-source library: library latency divided by each kernel's latency, higher is better, dashed line at the library's own 1×. The agent side takes, for each shape, the fastest of the MLSys'26 contest kernel and the publicly released agent kernels; parentheses name the system(s) that were the fastest across the operator's shapes. A blue bar longer than the orange bar means JustGoal is faster than the agent kernel. "—" indicates that the operator has no agent kernel in the comparison. Some experiments were completed with other models due to model fallback; see Appendix C. The DSA top-k indexer candidates passed correctness retests on limited inputs, but their historical baselines have not been aligned with the candidates' inputs and timing protocol, so this operator is excluded from the figure; per-shape latencies are retained in Appendix C.

Against kernel agents. The orange bars are the kernel agents' kernels against the same open-source library, taking for each shape the fastest of the MLSys'26 contest kernel and the publicly released kernels of the four agent systems; the ratio of the blue bar to the orange bar is therefore JustGoal's speedup over the agent kernel. Nine of the 10 plotted operators have a comparable agent kernel (FA4 backward has none); JustGoal is faster than the fastest agent kernel on five of them, ties on MoE FP8 (1.79× versus 1.80×), and falls behind on FA4 forward (0.77× versus AVO's 1.18×), DSA forward (1.66× versus 1.72×), and Kimi Delta Attention (0.99× versus CAKE's 1.27×). These agent kernels were produced by each system with its own choice of model and effort, so this comparison is between the kernels themselves; what the harness itself contributes under the same model is answered in Sections 4.3 and 4.4.

Relation to the released KDA-0.5 kernels. On the nine DSA, GDN, and MoE shapes, the kernels released in the KDA-0.5 repository reach a geometric mean of 1.29×, and single-shape JustGoal reaches 1.20× on the same nine shapes, close to the release; per-shape values

are in Appendix C. Our reproduction of its harness with Fable 5.1 reaches 0.91× (same-GPU paired protocol, Table 7 in the appendix); the model and effort behind the release are not public, and this gap bounds the fidelity of the reproduction.

The comparison with human-written libraries needs no further argument: JustGoal is faster on 8 of the 10 operators, and faster than the fastest agent release on 5 of the 9 operators that have one. Whether specialized machinery adds value beyond this result is a separate question. The next section first examines how the gap between full configurations changes across models; the following scope-matched comparison directly measures the mechanism gain.

4.3 Configuration Gaps Across Models

Configuration. Four operators (FA4 backward, DSA forward, GDN forward, MoE FP8), three shapes each, 12 shapes in all; every run is repeated with each of the three models under the same 24-hour budget. This section and Section 4.4 compare four configurations:

- *JustGoal, single shape*: one session optimizes one shape. 12 shapes × 3 models, 36 sessions.
- *JustGoal, whole definition*: the same prompt, with the goal changed to every workload of the operator's FlashInfer-Bench definition (23 for DSA, 100 for GDN, 19 for MoE FP8, 3 for FA4 backward); the session may dispatch by shape, and the final solution must pass the official evaluator on every workload. 4 operators × 3 models, 12 sessions.
- *KDA-0.5 reproduction*: the whole-definition scope, run through the KDA-0.5 harness with the same three models. KDA-0.5 has not fully released its prompts, so we follow its repository and technical report (MIT HAN Lab, 2026; HAN Lab Kernel Mafia, 2026): the humanize2 `flame_chase` two-chaser workflow, each turn a fresh session sharing the git workspace; the KernelWiki, NCU report parsing, and `iket-profiling` skills; the public KDA 0.1 contest prompt as the task and FlashInfer-Bench as the evaluator. 4 operators × 3 models, 12 runs.
- *KDA-0.5 reproduction, single shape*: the same KDA-0.5 harness; as in JustGoal, single shape, each run optimizes only one shape. Run with Fable 5.1 only: 12 shapes, 12 runs.

The reference kernel is the FA4 library kernel for FA4 backward and the MLSys'26 contest kernel for the other three operators. Every implementation follows the retest of the public repository; an implementation whose released version failed the retest and passed after repair takes the repaired version's timing. Two runs violated the task constraints and are listed as is: the Opus 4.8 JustGoal run on DSA heavy load read the reference implementation, and the Opus 4.8 KDA-0.5 reproduction on MoE called FlashInfer library kernels. The kernels released in the KDA-0.5 repository were produced with an unknown model and are therefore not part of the same-model comparison; their comparison with JustGoal is in Section 4.2.

Results. The first three configurations exist for all three models. Figure 3 gives, per operator, the geometric mean speedup of single-shape JustGoal and of the KDA-0.5 reproduction over the reference kernel across models; per-shape values are in Table 7 in the appendix. The two differ in task scope, so this section only examines the configuration gap; the scope-matched comparison is in Section 4.4.

The gap between configurations changes across models. Over Table 7, single-shape JustGoal's geometric mean speedup over the reference kernel rises from 0.51× with Opus 4.8 to 0.67× with Opus 5 and 1.18× with Fable 5.1 (12 shapes each), and the same-model KDA-0.5 reproduction from 0.65× to 0.87× and 0.97×: from Opus 4.8 to Fable 5.1, JustGoal improves by about 2.3× and the KDA-0.5 reproduction by about 1.5×. JustGoal trails the same-model KDA-0.5 reproduction with Opus 4.8 and Opus 5 and overtakes it at Fable 5.1, where its geometric mean exceeds 1× on all four operators while the KDA-0.5 reproduction stays below 1× on GDN. The two configurations use different task scopes, so this comparison

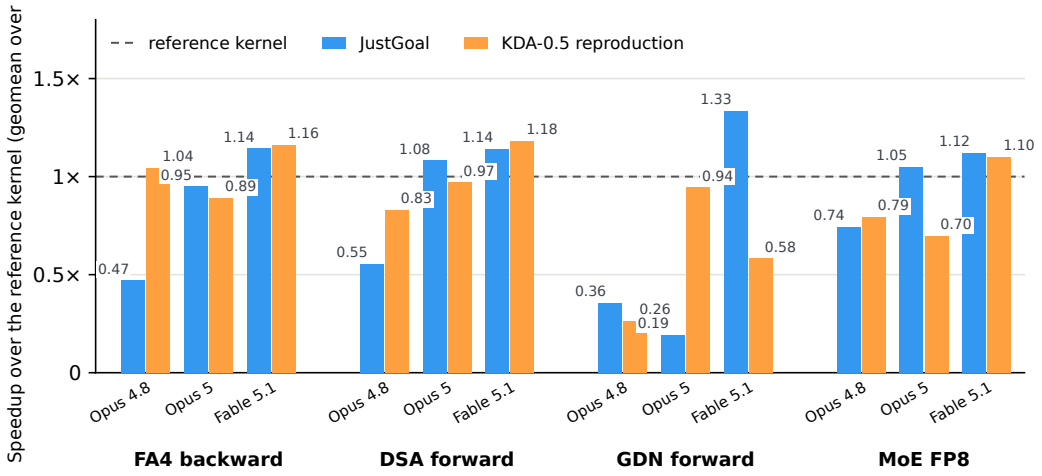


Figure 3: Speedup of single-shape JustGoal and the KDA-0.5 reproduction (whole definition) over the reference kernel across model generations on four operators; each bar is the geometric mean over the operator's shapes, the dashed line is the reference kernel. Parentheses give the number of shapes where fewer than three.

Table 2: The 2×2 of harness and task scope with Fable 5.1. The first four columns are the geometric mean over the operator's three shapes of the speedup over the reference kernel (higher is better); the single-shape columns are same-GPU paired re-measurements, the whole-definition columns official-evaluator timings; per-shape values are in Table 6. The last four columns are ratios of the first four: harness effect = KDA-0.5 ÷ JustGoal at equal scope, scope effect = whole definition ÷ single shape under the same harness. The last row takes the geometric mean over the 12 shapes first, then the ratios.

Operator	single shape		whole definition		harness effect		scope effect	
	JustGoal	KDA-0.5	JustGoal	KDA-0.5	single	whole	JustGoal	KDA-0.5
FA4 backward	1.14×	1.28×	0.79×	1.18×	1.12×	1.49×	0.69×	0.92×
DSA forward	1.14×	0.94×	0.74×	1.20×	0.83×	1.63×	0.65×	1.27×
GDN forward	1.33×	1.13×	0.69×	0.57×	0.84×	0.83×	0.52×	0.51×
MoE FP8	1.12×	1.17×	0.64×	1.11×	1.05×	1.73×	0.57×	0.94×
Geomean	1.18×	1.12×	0.71×	0.97×	0.95×	1.37×	0.60×	0.87×

only shows that their relative performance changes with the model; it cannot attribute the gap to any mechanism in the harness. Each cell is one run and three models give only three points, so this is an observed cross-model change rather than a monotonic per-generation trend. The Fable 5.1 2×2 in the next section provides the equal-scope evidence about mechanism gains.

4.4 Harness and Task Scope: The Fable 5.1 2×2

With Fable 5.1, the four configurations form the 2×2 {JustGoal, KDA-0.5 harness} × {single shape, whole definition}. Table 2 gives the four cells and the two marginal effects per operator, and Table 6 in the appendix the per-shape values. This 2×2 is the core controlled comparison for measuring the marginal value of explicit expert mechanisms. We examine the increment at equal scope and across scopes.

No increment at equal scope. At single shape, JustGoal reaches a geometric mean of 1.18× and the KDA-0.5 harness 1.12×; each is faster on 6 of the 12 shapes, and the harness effect is 0.95×, between 0.83 and 1.12 per operator. The harness is faster on all three FA4 backward shapes (1.35, 1.35, 1.16 versus 1.09, 1.27, 1.08) and on MoE single token and

large prefill; JustGoal is faster on all six DSA and GDN shapes and on MoE medium batch. The two paired re-measurements of the KDA-0.5 single-shape run on GDN long sequence gave 0.84× and 1.01×; the table takes their equal-weight aggregate. Under the same model and the same scope, adding all of KDA-0.5's machinery produces no overall gain; the gap varies by operator, FA4 backward being one where the harness leads, and the cross-kernel concurrency scheme ruled out by the workflow in the GDN case of Section 4.6 a concrete instance where JustGoal leads.

An increment at whole definition. With the goal widened to every workload, JustGoal's geometric mean drops to 0.71× and the KDA-0.5 harness reaches 0.97×, a harness effect of 1.37×, above 1 on every operator except GDN (0.83×). Whole-definition JustGoal crosses the reference kernel on no operator, whereas the same harness crosses it on FA4 backward, DSA, and MoE. This is the increment on the harness side that the data support.

Scope effect. Widening the scope from one shape to the whole definition takes JustGoal down to 0.60× and the KDA-0.5 harness to 0.87×; on GDN long sequence, JustGoal falls from 1.32× to 0.70×. The difference between the two scope effects is the part of the partitioning that the harness's multi-session loop and per-shape dispatch do for the model. Even so, the 0.97× the harness reaches at whole definition stays below the 1.18× of single-shape JustGoal. The scope effects of all three models are in Table 9 in the appendix.

Interpretation. On these Fable 5.1 tasks, the specialized harness produces no overall increment at equal scope on a single shape, and an increment appears only at whole definition. The observed gain is therefore associated with the broader task scope rather than higher overall performance under a single goal. The scope effect measures how well the agent can, on its own, partition a definition's workloads into implementations and optimize each. The current model is not yet good enough at this; the harness does part of the partitioning for the model and recovers part of the loss, but not all of it. This conclusion is limited to Fable 5.1: the 2×2 is complete only for this model, while the other two models have three cells each.

4.5 Ablation Study

Fable 5.1 on three shapes, FA4 backward varlen, DSA forward heavy load, and GDN forward long sequences, with default JustGoal as the reference and one mechanism added at a time: a link to the KernelWiki repository in the prompt (Kernel Wiki); an instruction to use NCU for performance analysis (NCU); a report-parsing skill on top of the NCU instruction (NCU skill); and the goal prefix replaced by the humanize2 `flame_chase` two-agent workflow (flame chase).

Table 3: Speedup of the kernel produced under each configuration over the reference kernel (higher is faster). JustGoal is the default; the best per row is bold, and a superscript $^{+}/^{-}$ marks a configuration faster/slower than JustGoal on that shape. Values come from the same-GPU paired re-measurement; an implementation not in the paired re-measurement, or whose released source was repaired after the retest, takes its cold-L2 kernel span in the retest repository over the independent timing of the same reference kernel.

Shape	JustGoal	Kernel Wiki	NCU	NCU skill	flame chase
FA4 backward varlen	1.08×	0.91×$^{-}$	**1.28**×$^{+}$	1.19×$^{+}$	0.69×$^{-}$
DSA forward heavy	0.90×	1.12×$^{+}$	**1.17**×$^{+}$	1.09×$^{+}$	1.10×$^{+}$
GDN forward long	1.32×	1.14×$^{-}$	**1.65**×$^{+}$	1.48×$^{+}$	1.36×$^{+}$

Three of the six NCU-family runs failed the retest as released (NCU skill on FA4 backward varlen and DSA, NCU on GDN) and passed after repair; Table 3 carries the repaired timing, and the other nine runs all passed. NCU beats the default on all three tasks and NCU skill falls below NCU on all three, but every task has one NCU-family run that failed as released: the speed comes with a higher failure rate, and nothing shows a dedicated NCU skill is worth writing. Every arm builds on JustGoal and runs no official evaluator in the loop, so

Table 4: Ablation of the accuracy verification: directories whose released version passes the retest / directories retested, per JustGoal configuration; the correctness check is built by the model in every case. The retest is at the audited commit of the public repository; a directory that failed as released and passed only after repair counts as failing. The other 7 operators are those of Appendix C. Runs that violated the task constraints but passed the retest count as passing. Whole definition has five directories per model because FA4 backward has a fixed-length and a varlen definition; the Opus 5 failure is the MoE solution's Triton launch-parameter mismatch.

Configuration	Opus 4.8	Opus 5	Fable 5.1
JustGoal single shape, 4 main operators	11/12	9/12	12/12
JustGoal single shape, other 7 operators	1/3	4/7	22/26
JustGoal whole definition	5/5	4/5	5/5

these failures are cases of the soft accuracy constraint that the accuracy verification below examines: the model-built check lets incorrect kernels through, and in this ablation it did so only in the NCU-family runs. Kernel Wiki beats the default only on DSA; flame chase, the humanize2 workflow KDA-0.5 depends on, trails it clearly on FA4 backward varlen and is slightly above it on DSA and GDN. On these Fable 5.1 tasks, making tools available is useful, but specialized mechanisms built around them do not consistently add gains. Single-mechanism ablations measure marginal effects on the default, and mechanisms interact: the full KDA-0.5 harness reaches 1.16× on FA4 backward varlen at single shape (Table 6 in the appendix), whereas its flame chase arm alone reaches 0.69×. Flame chase, Kernel Wiki, and NCU skill bring no consistent gain on these single-goal tasks. The full KDA-0.5 harness's whole-definition gain cannot establish that any one of these mechanisms is effective because the harness also includes multi-session execution and per-shape dispatch. The single-mechanism ablations cover only Fable 5.1 and therefore do not establish how their returns change with model capability.

Accuracy verification. This comparison contrasts two kinds of accuracy constraint. Without a hard constraint, JustGoal only asks for accuracy verification in the prompt and says how to set the tolerance; the model writes the check itself, and nothing outside it stops a kernel that misses the tolerance from becoming the final version. With a hard constraint, as in the KDA-0.5 reproduction, the harness runs the official evaluator in the loop and a failing kernel cannot be submitted. Under the hard constraint the evaluator decides: all 12 reproductions with Opus 5 and Fable 5.1 pass the retest, and the 3 Opus 4.8 DSA kernels fail for not returning lse, a case the evaluator did not catch at the time. Without it, Table 4 counts, per JustGoal configuration, the directories whose released version passes the retest of the public repository: the model can build its own numerical check, yet some kernels still fail accuracy. Single-shape JustGoal passes 11/12, 9/12 and 12/12 on the four main operators, 1/3, 4/7 and 22/26 on the other seven, and the whole-definition runs 5/5, 4/5 and 5/5. The failures are accuracy, missing outputs, build or runtime errors. With the stronger model the model-built check approaches the pass rate of the hard constraint but does not replace it.

4.6 Optimization Case Study

We use GDN (Gated DeltaNet) forward on long sequences (8192 tokens in 20 sequences of length 15 to 3371) to show how JustGoal crosses the contest kernel from scratch. The reference is the MLSys'26 contest kernel from the KDA-0.5 submission, which the agent could only call as a black box; the audited trajectory never reads its source. Fable 5.1 ran 8.9 hours without intervention. Latencies are the session's own L2-flushed kernel span; the reference is 125.1 μs. Appendix B gives the full record and a second case, FA4 backward GQA.

The trajectory (Figure 4) has three stages, each ending when the agent measured that its toolchain had run out of headroom. Triton started at 302 μs (0.41×) and the agent put its floor near 146 μs. After microbenchmarking `mma.sync` on B200, it hand-wrote CUDA

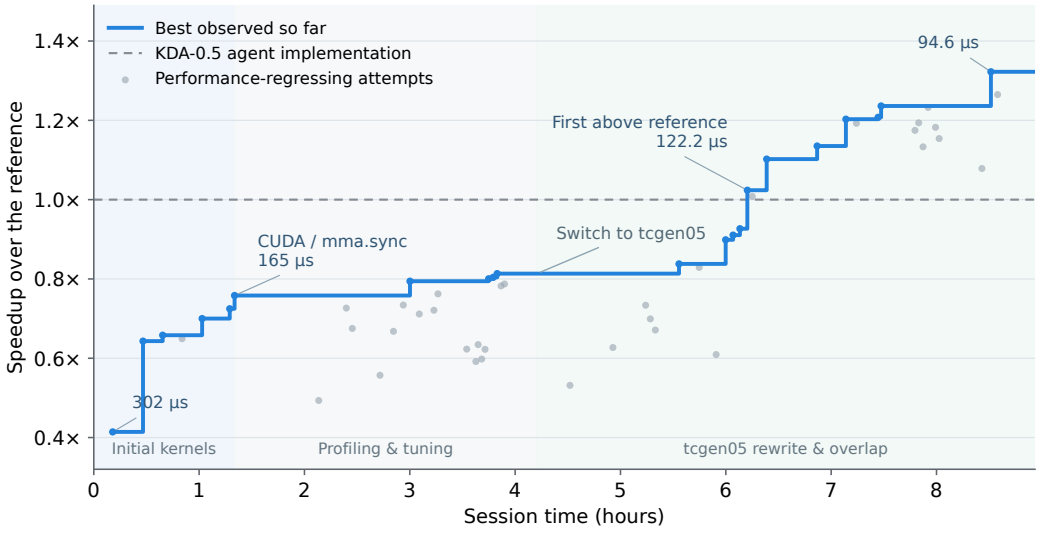


Figure 4: Optimization process of JustGoal (Fable 5.1) on GDN forward, long sequences. The staircase shows the session's best observation so far. The 34 gray points represent log-reviewed change batches that did not improve the record, not all attempts: overlap mode is used when available, otherwise serial or the original default mode; within-mode repeats use median latency. Failed patches, repeat-only batches and batches with additional timing or execution concerns are omitted from gray points. Time denotes tool-result receipt; shading marks strategy phases. The minimum observation is 94.6 μs; the final version measures 95.6–97.3 μs in three independent processes. The main table's same-GPU paired re-measurement uses a different protocol and cannot be spliced with this trace.

kernels and, guided by NCU stall counters and in-kernel cycle counts, reached 165 μs at 1.3 hours; further overlap and cluster work only reached 153.8 μs. At 4.2 hours it judged the path near the shared-memory bandwidth limit and moved the recurrence to B200's tcgen05 tensor cores, checking descriptor encodings and TMEM layouts with probe kernels. The first tcgen05 version was slower (235 μs); warp specialization, fusing the transposed O product into the recurrence's MMA group, and running the intra-chunk kernel on the remaining SMs crossed the reference at 6.2 hours. The final version measures 95.6–97.3 μs over three processes (1.29–1.31×; 1.32× under the paired re-measurement), with errors within 1.30 times the reference's.

Profiling, toolchain switching and hardware probing were all the agent's own choices. The prompt names no profiler, supplies no domain corpus and prescribes no stages, whereas KDA-0.5 encodes the same behaviors as NCU and in-kernel tracing skills, KernelWiki and three-stage prompts (Table 1). Given only the goal, the model carried out the expert workflow on its own.

5 Discussion

The effectiveness of specialized machinery should be judged by its measured gain over a goal-only baseline using the same model, rather than presumed from system complexity or results on earlier models. Across three models, the gap between single-shape JustGoal and whole-definition KDA-0.5 changes with the model. Because their task scopes differ, this comparison describes a configuration gap rather than a causal mechanism gain. In the equal-scope Fable 5.1 comparison, the full harness produces no overall gain on single shapes, and single-mechanism ablations show no consistent gain from domain knowledge or workflows. The harness still helps at whole definition, showing that the current model is not yet good enough at organizing a broader task scope on its own. New agent designs should measure every added mechanism against a goal-only baseline with the same frontier

model and task scope, and repeat that measurement when the model changes. Goal-only generation and final acceptance should be separated. JustGoal does not include the official evaluator in its optimization loop, but every final kernel reported in this paper undergoes independent validation. Model-built checks still let incorrect kernels through (Table 4); in the ablation, this occurred in three of the six NCU-family runs while every other arm passed (Section 4.5). Independent acceptance is necessary for credible evaluation, but it is not kernel-specific machinery in the JustGoal generation process.

Task scope is also critical. The scope effect shows that making one implementation cover all workloads of an operator is a generality tax, and current models cannot yet partition the workloads well enough on their own to avoid it. This leaves two paths. One is to make the agent's own partitioning ability a design goal of kernel agents and use the scope effect in Table 2 to measure it. The other is deep customization: in high-value scenarios with fixed configurations, no single implementation has to cover every workload, the generality tax disappears entirely, and the single-shape column of Table 6 is the level reached once it does. We call the latter Forge Engineering. Each implementation optimizes performance only for its target configuration but must remain correct for any input of that configuration; the costs are optimization sessions that grow with the number of configurations, plus per-configuration verification.

These results also invite us to reconsider the human role in kernel development. We envision that, when models can construct evaluation pipelines and iteratively optimize kernels on their own, the focus of human work may shift from writing kernels and prescribing optimization steps to defining problems and constraints, validating results, and judging whether an optimization meets practical needs. Domain expertise can serve as a reference for a model's autonomous decisions or be encoded into an optimization workflow for the model to follow. How to provide effective guidance while preserving the model's freedom to adapt its strategy and explore alternative solutions is an important direction for further research.

6 Limitations

Our experiments focus on selected operators and shapes on NVIDIA B200 GPUs; the applicability of our findings to other hardware and tasks remains to be validated. Comparisons with publicly released kernels concern the kernels themselves; the systems use different models and optimization budgets. The same-model comparisons in Section 4.3 separately examine the harness. KDA has released its general workflow, base prompts, and related skills, but the KDA-0.5 results repository primarily provides the final kernels and retest code rather than the complete prompts, workflow configuration, and search process used for that version. Our reproduction can therefore only follow the available public materials and may differ from the original system. KDA-0.5 also bundles domain priors with scope-organization mechanisms, and we do not ablate these two groups separately. Their distinction in Section 4.4 is our interpretation of the data.

Our correctness validation remains limited by test coverage. Except for FA4 backward, which follows the official test criteria, numerical accuracy checks for all other kernels are aligned with FlashInfer-Bench. However, passing error-tolerance checks on a finite set of test inputs does not guarantee correctness for all valid inputs and execution conditions. The reported performance results should therefore be understood as results obtained after passing the adopted validation procedures, rather than a comprehensive guarantee of production readiness.

7 Conclusion

We first identify a change in current model capability. Given only objectives and constraints, without predefined kernel-specific evaluation infrastructure, domain knowledge, or search workflows, a general-purpose agent can construct its own evaluation pipeline and iteratively optimize kernels. On NVIDIA B200 GPUs, JustGoal is faster than human-maintained open-source libraries on 8 of 10 operators and faster than the fastest specialized-agent release on 5 of 9. Without task-time injection of kernel-specific knowledge or optimization

workflows, current frontier models can establish evaluation feedback, identify performance bottlenecks, adapt optimization strategies, and cross the performance line represented by human-maintained implementations on a range of operators.

We further propose an evaluation principle that does not depend on the current level of model capability. JustGoal is not a new agent architecture, but the experimental configuration used to observe the performance level of a goal-only setup and measure the marginal value of explicit expert mechanisms. Specialized machinery should be considered effective only when it clearly outperforms this baseline. When model capabilities change, the returns of existing mechanisms should be measured again rather than inherited from earlier results. The current model still benefits from a harness at broader task scope; whether future models do so should also be answered by a scope-matched comparison. We hope future work will not only apply new models to existing systems but also use a goal-only configuration under the same model and task scope as the baseline, reexamine each component's contribution against it, and determine which components to retain, adapt, or remove.

References

AKO Team. AKO: Agentic kernel optimization. `https://github.com/TongmingLAIC/AKO`, 2026a.

AKO Team. AKO4ALL: Agentic kernel optimization for all. `https://github.com/TongmingLAIC/AKO4ALL`, 2026b.

AKO Team. AKO4X: Agentic kernel optimization, advanced and extensible. `https://github.com/TongmingLAIC/AKO4X`, 2026c.

Anthropic. Claude code. `https://claude.com/claude-code`, 2025.

Shiyi Cao, Ziming Mao, Joseph E. Gonzalez, and Ion Stoica. K-Search: LLM kernel generation via co-evolving intrinsic world model, 2026. arXiv:2602.19128.

Tri Dao, Daniel Y. Fu, Stefano Ermon, Atri Rudra, and Christopher Ré. Flashattention: Fast and memory-efficient exact attention with IO-awareness. In *Advances in Neural Information Processing Systems*, 2022.

Tri Dao et al. Flashattention-4: Algorithm and kernel pipelining co-design for asymmetric hardware scaling. In *Proceedings of Machine Learning and Systems (MLSys)*, 2026. arXiv:2603.05451.

DeepSeek-AI. DeepGEMM: Clean and efficient FP8 GEMM kernels with fine-grained scaling. `https://github.com/deepseek-ai/DeepGEMM`, 2025a.

DeepSeek-AI. FlashMLA: Efficient multi-head latent attention kernels. `https://github.com/deepseek-ai/FlashMLA`, 2025b.

DeepSeek-AI. Deepseek-v3.2: Pushing the frontier of open large language models, 2025c. arXiv:2512.02556.

Weihua Du, Jingming Zhuo, Yixin Dong, Andre Wang He, Weiwei Sun, Zeyu Zheng, Manupa Karunaratne, Ivan Fox, Tim Dettmers, Tianqi Chen, Yiming Yang, and Sean Welleck. AdaExplore: Failure-driven adaptation and diversity-preserving search for efficient kernel generation, 2026. arXiv:2604.16625.

HAN Lab Kernel Mafia. Developing GPU kernels with agentic loops. `https://github.com/mit-han-lab/mlsys2026-flashinfer-contest`, 2026. Technical report, with the per-operator phase prompts used for the competition runs.

METR. Measuring automated kernel engineering. `https://metr.org/blog/2025-02-14-measuring-automated-kernel-engineering`, 2025.

MIT HAN Lab. Kernel design agents: An agent-centric workflow for high-performance CUDA kernels. `https://github.com/mit-han-lab/kernel-design-agents`, 2026.

NVIDIA. AVO: Agentic variation operators for autonomous evolutionary search, 2026. arXiv:2603.24517.

OpenAI. Codex. `https://openai.com/codex`, 2025.

Anne Ouyang, Simon Guo, Simran Arora, Alex L. Zhang, William Hu, Christopher Ré, and Azalia Mirhoseini. Kernelbench: Can LLMs write efficient GPU kernels?, 2025. arXiv:2502.10517.

Sakana AI. The AI CUDA Engineer: Agentic CUDA kernel discovery, optimization and composition. `https://sakana.ai/ai-cuda-engineer/`, 2025.

Jay Shah, Ganesh Bikshandi, Ying Zhang, Vijay Thakkar, Pradeep Ramani, and Tri Dao. Flashattention-3: Fast and accurate attention with asynchrony and low-precision. In *Advances in Neural Information Processing Systems*, 2024.

Zihao Ye, Yingyi Huang, Hongyi Jin, et al. CAKE: Compiler-agent co-design for frontier kernel evolution, 2026. arXiv:2608.12629.

Yunxiang Zhang, Ping Yu, Jianyu Wang, Max Fan, Julian Reed, Azalia Mirhoseini, and Will Su. KernelBench-Verified: Do LLM-generated kernels actually beat PyTorch?, 2026. arXiv:2607.16241.

Zijian Zhang, Rong Wang, Shiyang Li, Yuebo Luo, Mingyi Hong, and Caiwen Ding. CudaForge: An agent framework with hardware feedback for CUDA kernel optimization, 2025. arXiv:2511.01884.

A Original Chinese Prompt

Figure 1 is an English translation; the box below is the original Chinese prompt given to the agent.

> 这是我们要的目标算子（算子 config），请先在 B200 上测量它的 device time 时间。从 0 开始，绝对不看或抄写这个 ref 的代码实现，写出一个至少达到他 1.2X 倍快 device time 速度的 kernel 实现。
>
> 1. 请使用一个独立的干净的环境，不要用已有的 skills，memory，或者其它本机文件。
> 2. 请务必确保正确性。最好通过先写一个 fp32 的 naive 慢实现，再和 ref 比他们两的差这样可以获得一个下限容差范围再乘个值，然后用这个容差来对比你的实现和 fp32 慢实现的。
> 3. 不要问问题，用 `--dangerously-skip-permissions` 模式。
> 4. GPU 请使用 B200 集群。

B Optimization Trajectories in Detail

This appendix gives the full record behind Section 4.6: a second case, FA4 backward GQA, and the step-by-step trajectory of the GDN forward case summarized there. Both have complete trajectories, and all per-version latencies are the sessions' own measurements.

B.1 FA4 Backward GQA

We use FA4 backward GQA ($B = 8$, $N = 4096$, $H = 16$, $H_{\mathrm{KV}} = 2$, $D = 128$, causal, bf16). The reference is FlashAttention-4 by Tri Dao et al. (Dao et al., 2026), which the agent can invoke only as a black box through its public API. We audited the complete trajectory and confirmed that the agent did not read its source code. Fable 5.1 chose Triton Gluon to implement the kernel and completed the task in four hours without human intervention.

The agent first measured the reference kernel's device time and wrote a naive fp32 implementation as a correctness oracle. Its first kernel took 2771 μs and passed correctness checks on the first attempt, but was approximately 80% slower than the reference. The agent then profiled stall reasons with NCU, added ablation switches and `%clock` instrumentation to measure cycles per stage. It found that MMA alone required only about 830 μs, while softmax and atomic accumulation of dQ accounted for most of the time. By reusing TMEM and switching dQ accumulation to TMA bulk reduction, it reduced latency to approximately 1320 μs after 2.5 hours.

Over the next 1.4 hours, more than a dozen new versions improved the best latency only to 1278 μs (Figure 5). The agent wrote microbenchmarks and measured TMA reduction throughput of only 25–30 B/clk per SM, with roughly 2800 cycles required to release a staging buffer. Disabling dQ reduction reduced the main kernel's latency to 931 μs. These observations pointed to the same cause: reduction reused the staging buffer for dO, blocking dO loads. The agent therefore allocated a dedicated 32 KB staging buffer for reduction and moved dO loads to drain warps using `cp.async`, leaving TMA exclusively for reduction. This change reduced latency to 1166 μs in a single step. Under the unified cold-L2 paired re-measurement, the final kernel runs in 1321 μs against 1676 μs for FA4, a speedup of 1.27×. The final kernel accumulates dQ in FP16 and rounds dS to BF16, so its internal numerics differ from FA4; its dQ, dK and dV all pass the gradient tolerance of the upstream FA4 tests, with errors at 20–33% of the allowed budget. The backward case is verified with this upstream tolerance rather than the reference-error-based tolerance used for the forward cases.

Throughout this process, the agent independently used NCU, ablations, and hardware microbenchmarks to diagnose bottlenecks and adjusted the kernel structure accordingly, ultimately producing an operator that outperforms the expert implementation.

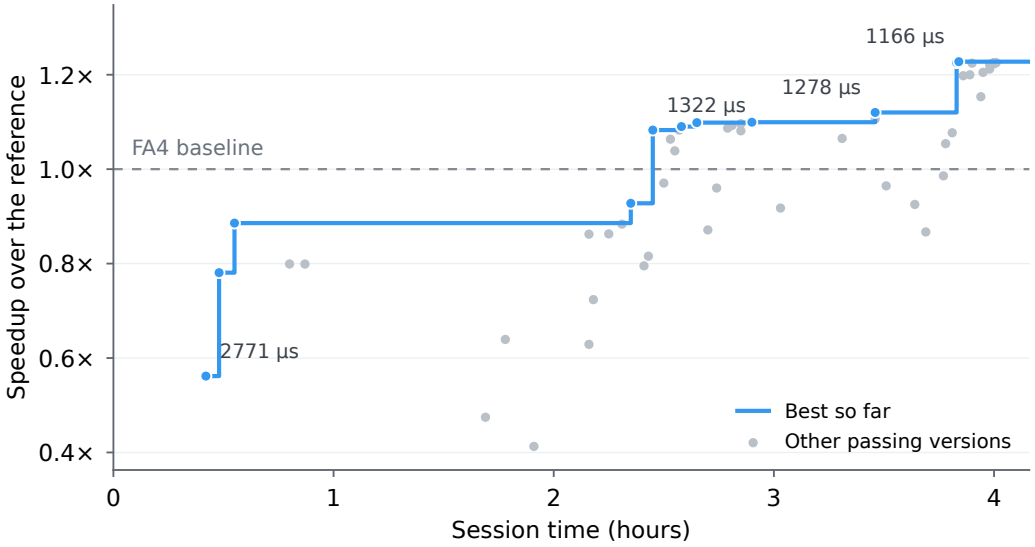


Figure 5: Optimization trajectory of JustGoal (Table 5.1) on FA4 backward GQA. Per-version latencies are the original session's own measurements and show the optimization process; the same-GPU paired re-measurement of the final kernel is in the main table, and the two protocols cannot be spliced.

B.2 GDN Forward, Long Sequences

The workload, reference and measurement protocol are those of Section 4.6; this is the full record of the 8.9-hour run.

The agent first wrote a Triton version that passed correctness at once but took 302 μs, 0.41× the reference (Figure 4). Sweeping tile and warp configurations showed a fixed per-step overhead in the recurrence kernel and register spills in the intra-chunk kernel, with a floor of about 146 μs for the Triton path, so it abandoned Triton, first wrote microbenchmarks for `mma.sync` throughput and dependency latency on B200, and then hand-wrote two CUDA C++ kernels with `mma.sync` and `ldmatrix`, reaching 194.5 μs at 0.5 hours. It then checked spills with `cuobjdump`, located hot spots with NCU stall and source counters, and counted cycles per phase inside the kernel, bringing the recurrence from 4025 to 1993 cycles per step and the total to 165 μs at 1.3 hours. The next two hours gained little: overlapping the three kernels on two streams with per-block flags made things slower; several launch failures and deadlocks were debugged with compute-sanitizer; thread-block clusters with TMA multicast let 8 CTAs share one broadcast of the W/K tiles; and `globaltimer` timelines of every CTA's work items showed a busy ratio of only 0.74 with about 5 μs gaps between items, so it switched to static round-robin assignment to remove the atomics. At 3.8 hours it stood at 153.8 μs, 0.81×.

At 4.2 hours the agent reported plainly that correctness was met but performance was not, gave a breakdown of where the time went, and judged that the `mma.sync` path was near the shared-memory bandwidth limit, so the recurrence had to move to B200's fifth-generation tensor cores (tcgen05, whose operands are read from shared memory by the tensor core and whose accumulators live in TMEM). It consulted the PTX ISA documentation and CUTLASS headers, wrote probe kernels to verify descriptor encodings, swizzles and TMEM layouts, and rewrote the recurrence kernel. The first tcgen05 version took 235 μs, about 3 μs per step, bottlenecked by 24 serial TMEM waits. Over the next three hours it dumped TMEM intermediates and rebuilt O in fp32 to localize a numerical error, fused the transposed O product into the recurrence's MMA group, microbenchmarked A operands from TMEM versus shared memory, split TMA production, MMA issue and output write-back into dedicated warps, and launched the recurrence first at the highest stream priority

with the intra-chunk kernel running on the remaining SMs. It first crossed the reference at 6.2 hours (122 μs) and reached 101 μs at 7.5 hours. Finally it made the intra-chunk kernel process one v head per CTA with the longest sequences first, reaching 94.6 μs at 8.5 hours; the final version measured 95.6 to 97.3 μs over three independent processes, 1.29–1.31× the reference, with output and state errors at 1.00 and 1.30 times the reference error. Under the same-GPU paired re-measurement it reaches 1.32× over the MLSys'26 contest kernel shipped with KDA-0.5.

C Per-Shape Latencies and Baseline Sources

Table 5 lists implementation latencies and baseline sources. The library column only lists human-maintained library implementations; the MLSys'26 contest kernels are listed separately and, together with the released agent kernels, form the kernel-agent baseline of Section 4.2, taking the faster of the two per shape. Some experiments were completed with other models due to model fallback: the medium- and large-batch DSA top-k indexer tasks used Opus 5.5, while the short and medium MLA paged prefill tasks and the medium- and large-batch GQA paged decode tasks used Opus 5. The target model for all other tasks was Fable 5.1. The original length distributions of four shapes were not recovered, so they were measured on the substitute distribution of an earlier round: GQA paged decode medium and large batch, Kimi Delta Attention packed mixed lengths, and MLA paged prefill large prefill.

The three DSA top-k indexer candidates passed correctness retests on their respective limited inputs, with cold-L2 kernel spans of 47.59, 3531.5, and 13.25 μs; the medium-batch result uses the repaired kernel, and the large-batch result uses signed weights. The three historical top-k baselines (AKO4X fast_split_v6 and v8, and Dogacel) compress scores to BF16 before selection and fail a strict FP32 membership check on 9 of their 18 input configurations; after a repair that keeps FP32 scores, all 18 pass. The agent kernel column gives the faster repaired AKO4X version, as a range over non-negative and signed weights; the repaired Dogacel measures 1.44–1.47, 15.24–24.46 and 18.02–27.43 μs. The library and contest columns remain historical values. None of these baselines was paired with the candidates on the same GPU, so the latencies are retained as audit records and are not used to compute speedups in Figure 2.

Absolute latencies from different measurement batches are not comparable; every speedup uses the baseline of its own pair. On GDN long sequences the contest kernel measured 209.5 μs in the batch paired with JustGoal and 124.9 μs in the independent baseline retest; across the batches of this appendix, the FA4 backward baseline spans 1371–1482, 1456–1676 and 1572–1706 μs on MHA, GQA and varlen MHA.

Table 5: Per-shape latency (μs, lower is better) and baseline sources. The library column only lists human-maintained library implementations; contest kernels are the MLSys'26 contest kernels and, together with the released agent kernels, form the kernel-agent baseline; the main text takes the faster of the two per shape. The JustGoal column is the candidate latency of the same-GPU paired re-measurement; an implementation not in it, or whose released source was repaired after the retest, takes the cold-L2 kernel span at the audited commit of the retest repository. Baselines measured under the same protocol follow; the remaining baselines await a unified rerun. A few shapes were completed with another model, and four shapes were measured on a substitute length distribution because the original one is missing; see the text.

Operator	Shape	JustGoal	Library	Contest kernel	Released agent kernel
FA4 backward	MHA	1357.9	1481.9 (FA4)	—	—
	GQA	1321.2	1675.5 (FA4)	—	—
	Variable-length MHA	1581.9	1705.6 (FA4)	—	—
FA4 forward	MHA (8,4096) non-causal	874.0	981 (FA4)	—	699 (AVO)
	MHA (1,32768) non-causal	6709.6	5930 (FA4)	—	5270 (AVO)
	MHA (8,4096) causal	544.0	460 (FA4)	—	395 (AVO)
	MHA (1,32768) causal	4188.9	3170 (FA4)	—	2690 (AVO)

Continued on next page

Operator	Shape	JustGoal	Library	Contest kernel	Released agent kernel
	GQA g8 (8,4096) non-causal	2277.2	1558 (FA4)	—	1336 (AVO)
	GQA g8 (1,32768) non-causal	13205.8	—	—	—
	GQA g8 (8,4096) causal	2501.4	960 (FA4)	—	749 (AVO)
	GQA g8 (1,32768) causal	6667.6	5750 (FA4)	—	5410 (AVO)
	GQA g4 (8,4096) non-causal	1767.0	1406 (FA4)	—	1347 (AVO)
	GQA g4 (1,32768) non-causal	15886.9	—	—	—
	GQA g4 (8,4096) causal	1003.0	929.5 (FA4)	—	751 (AVO)
	GQA g4 (1,32768) causal	8321.2	6030 (FA4)	—	5340 (AVO)
DSA forward	Heavy workload	11.42	13.71 (FlashInfer)	10.27 (MLSys'26)	10 (KDA-0.5)
	Small request	2.891	7.03 (FlashInfer)	4.037 (MLSys'26)	3 (KDA-0.5)
	Mixed lengths	7.115	11.07 (FlashInfer)	8.427 (MLSys'26)	7 (KDA-0.5)
DSA top-k indexer	Single request, short sequence	47.59	154.88 (DeepGEMM + FlashInfer)	2.35 (MLSys'26)	1.47–1.55 (AKO4X, FP32 fix)
	Medium batch	3531.5	315.59 (DeepGEMM + FlashInfer)	17.21 (MLSys'26)	17.39–17.70 (AKO4X, FP32 fix)
	Large batch, mixed lengths	13.25	313.28 (DeepGEMM + FlashInfer)	21.73 (MLSys'26)	24.24–24.49 (AKO4X, FP32 fix)
GDN forward	Long sequences	158.5	145.6 (FlashInfer)	209.5 (MLSys'26)	126 (KDA-0.5)
	Single short sequence	4.133	45.5 (FlashInfer)	5.936 (MLSys'26)	6 (KDA-0.5)
	Few long sequences	105.5	133 (FlashInfer)	131.6 (MLSys'26)	105 (KDA-0.5)
GDN decode	Single request	2.347	3.39 (FlashInfer)	2.35 (MLSys'26)	2.31 (AKO4X)
	Medium batch	4.725	5.56 (FlashInfer)	5.27 (MLSys'26)	4.75 (AKO4X)
	Large batch	11.91	13.66 (FlashInfer)	12.51 (MLSys'26)	12.79 (AKO4X)
Kimi Delta Attention	Packed mixed lengths	740.0	455.6 (FlashKDA)	—	391 (CAKE)
	Single fixed-length long sequence	454.9	658.7 (FlashKDA)	—	470 (CAKE)
	Packed equal lengths	506.7	545.1 (FlashKDA)	—	435 (CAKE)
MLA paged decode	Single request	6.320	15.98 (FlashInfer)	—	9.38 (AKO4X)
	Medium batch, mixed lengths	15.57	31.25 (FlashInfer)	—	17.36 (AKO4X)
	Large batch	57.58	77.5 (FlashInfer)	—	69.97 (AKO4X)
MLA paged prefill	Short prefill	4.981	22.886 (FA4)	—	10.04 (AKO4X)
	Medium prefill	23.65	34.73 (FA4)	—	22.29 (AKO4X)
	Large prefill, mixed lengths	430.6	553.0 (FA4)	—	2491 (AKO4X)
GQA paged decode	Single request	4.853	7.488 (FA4)	—	7.74 (AKO4X)
	Medium batch, short sequences	10.15	16.07 (FlashInfer)	—	11.55 (AKO4X)
	Large batch	60.22	69.61 (FA4)	—	63.47 (AKO4X)
MoE FP8	Single token	50.09	86.41 (FlashInfer)	57.15 (MLSys'26)	49.05 (KDA-0.5)
	Medium batch	296.9	377.41 (FlashInfer)	302.3 (MLSys'26)	278.29 (KDA-0.5)
	Large prefill	967.1	2544 (FlashInfer)	1173.6 (MLSys'26)	1042.65 (KDA-0.5)

C.1 Full Configuration Results for All Three Models

Table 6 gives the per-shape speedups of the four Fable 5.1 configurations of Section 4.4; Table 2 in the main text is its per-operator geometric mean. Table 7 gives the per-shape

Table 6: Per-shape speedup of the four configurations over the reference kernel with Fable 5.1 (higher is better); Table 2 is its per-operator geometric mean. The two single-shape columns are same-GPU paired re-measurements; the two whole-definition columns are the official-evaluator timing of the definition-level implementation on this shape. The fastest configuration in each row is bold. Last row: geometric mean.

Operator	Shape	JustGoal, single shape	KDA-0.5, single shape	JustGoal, whole definition	KDA-0.5, whole definition
FA4 backward	MHA	1.09×	**1.35**×	0.71×	1.14×
	GQA	1.27×	**1.35**×	0.93×	1.25×
	varlen MHA	1.08×	**1.16**×	0.75×	1.14×
DSA forward	heavy load	0.90×	1.05×	0.76×	**1.36**×
	small request	**1.40**×	1.03×	0.71×	0.94×
	mixed lengths	1.18×	0.78×	0.74×	**1.35**×
GDN forward	long sequence	**1.32**×	0.92×	0.70×	0.53×
	single short seq.	**1.44**×	1.41×	0.90×	1.03×
	few long seqs.	**1.25**×	1.10×	0.53×	0.35×
MoE FP8	single token	1.14×	**1.29**×	0.97×	1.24×
	medium batch	**1.02**×	0.90×	0.64×	0.98×
	large prefill	1.21×	**1.38**×	0.42×	1.11×
Geomean		**1.18**×	1.12×	0.71×	0.97×

Table 7: Full same-model results for all three models (the Fable 5.1 JustGoal column equals that of Table 6): speedup over the reference kernel (reference latency / measured latency, higher is better). Reference kernel: the FA4 4.0.0b19 backward kernel (open-source library) for FA4 backward; for the other three operators, the MLSys'26 contest kernel, i.e. the fixed snapshot shipped with the KDA-0.5 contest submission (MIT HAN Lab), which belongs to the kernel-agent baseline and is not the open-source library listed in Appendix C. Values come from the same-GPU paired re-measurement, each candidate alternating with its baseline on one B200; the baselines' absolute latencies differ per pair and are not listed. An implementation not in the paired re-measurement, or whose released source was repaired after the retest, takes the ratio of the reference kernel's independent cold-L2 kernel span to the candidate's cold-L2 kernel span at the audited commit of the retest repository (same protocol, another B200, not interleaved). The faster of JustGoal and the KDA-0.5 reproduction is bold for each model; the last row is the geometric mean of the per-shape ratio of single-shape JustGoal to the definition-level KDA-0.5 reproduction over the shapes both have. The two differ in task scope; the scope-matched comparison is Table 9. Two runs violated the task constraints and are listed as is: the Opus 4.8 JustGoal run on DSA heavy load read the reference implementation, and the Opus 4.8 KDA-0.5 reproduction on MoE called FlashInfer library kernels. The KDA-0.5 reproduction here is the paired re-measurement; Tables 6 and 8 use the official-evaluator timing. The Fable 5.1 FA4 MHA run fell back to Opus 4.8 for about one hour.

Operator	Shape	Reference kernel	Opus 4.8		Opus 5		Fable 5.1	
			JustGoal	KDA-0.5	JustGoal	KDA-0.5	JustGoal	KDA-0.5
FA4 backward	MHA	FA4	0.45×	**0.97**×	**0.96**×	0.87×	**1.09**×	1.09×
	GQA	FA4	0.42×	**1.17**×	**1.12**×	0.92×	**1.27**×	1.26×
	varlen MHA	FA4	0.55×	**1.01**×	0.79×	**0.88**×	1.08×	**1.14**×
DSA forward	heavy load	contest kernel	0.60×	**0.90**×	1.04×	**1.17**×	0.90×	**1.37**×
	small request	contest kernel	**0.96**×	0.68×	**1.07**×	0.73×	**1.40**×	0.91×
	mixed lengths	contest kernel	0.30×	**0.93**×	**1.14**×	1.07×	1.18×	**1.33**×
GDN forward	long sequence	contest kernel	**0.58**×	0.26×	0.14×	**1.02**×	**1.32**×	0.53×
	single short seq.	contest kernel	**0.38**×	0.30×	1.02×	**1.05**×	**1.44**×	1.09×
	few long seqs.	contest kernel	0.20×	**0.24**×	0.05×	**0.79**×	**1.25**×	0.34×
MoE FP8	single token	contest kernel	**1.09**×	0.96×	**1.18**×	0.83×	1.14×	**1.18**×
	medium batch	contest kernel	0.57×	**0.80**×	**0.86**×	0.67×	**1.02**×	1.01×
	large prefill	contest kernel	**0.65**×	0.65×	**1.12**×	0.60×	**1.21**×	1.12×
single-shape JustGoal / KDA-0.5 repro., geomean			0.79× (12 shapes)		0.78× (12 shapes)		1.22× (12 shapes)	

speedups of three configurations under all three models; its Fable 5.1 JustGoal column equals that of Table 6. Table 8 gives the operator-level geometric means of the three configurations under all three models; the two effects of Table 9 are ratios of it.

Table 9 reduces the results of the three models to three ratios, each dividing the operator's geometric means over three shapes: scope effect = whole definition ÷ single shape, below 1 is

Table 8: Operator-level results of the three configurations. Geometric mean over three shapes of the speedup over the reference kernel (higher is better); the reference kernel is that of Table 6; the two effects of Table 9 are ratios of this table. The single-shape column is computed from the corresponding three cells of Table 7; the whole-definition and KDA-0.5 reproduction columns were re-measured on one B200 with the official evaluator (CUPTI kernel span, cold L2, median) in at least two alternating rounds. The faster of the last two columns is bold in each row. An implementation whose released source was repaired after the retest takes the repaired version's timing from the retest repository. The Opus 4.8 KDA-0.5 reproduction on MoE called FlashInfer library kernels and is listed as is. Per-workload latencies and the correctness re-check are in Appendix C.2. The last two columns await the same-GPU paired re-measurement.

Operator	Model	JustGoal, single shape	JustGoal, whole definition	KDA-0.5 reproduction
FA4 backward	Opus 4.8	0.47×	0.50×	**1.07**×
	Opus 5	0.95×	0.83×	**0.96**×
	Fable 5.1	1.14×	0.79×	**1.18**×
DSA forward	Opus 4.8	0.55×	0.45×	**0.67**×
	Opus 5	1.08×	0.96×	**1.01**×
	Fable 5.1	1.14×	0.74×	**1.20**×
GDN forward	Opus 4.8	0.36×	0.13×	**0.18**×
	Opus 5	0.19×	0.64×	**0.92**×
	Fable 5.1	1.33×	**0.69**×	0.57×
MoE FP8	Opus 4.8	0.74×	0.32×	**0.63**×
	Opus 5	1.05×	0.43×	**0.69**×
	Fable 5.1	1.12×	0.64×	**1.11**×

Table 9: Scope effect, harness effect, and gap to the single-shape ceiling per operator under the three models; last row: geometric mean.

Operator	Opus 4.8			Opus 5			Fable 5.1		
	scope	harness	vs. single	scope	harness	vs. single	scope	harness	vs. single
FA4 backward	1.07×	2.13×	2.27×	0.87×	1.16×	1.01×	0.69×	1.49×	1.03×
DSA forward	0.81×	1.50×	1.21×	0.89×	1.05×	0.93×	0.65×	1.63×	1.05×
GDN forward	0.37×	1.41×	0.52×	3.32×	1.45×	4.81×	0.52×	0.83×	0.43×
MoE FP8	0.44×	1.97×	0.86×	0.41×	1.59×	0.66×	0.57×	1.73×	0.99×
Geomean	0.61×	1.72×	1.05×	1.02×	1.30×	1.32×	0.60×	1.37×	0.82×

the loss from widening the goal; harness effect = KDA-0.5 reproduction ÷ whole definition, above 1 means that the KDA-0.5 configuration is faster at equal scope; vs. single = KDA-0.5 reproduction ÷ single-shape JustGoal. In 11 of the 12 operator-model combinations, whole-definition JustGoal trails the same-model KDA-0.5 reproduction, and it crosses the reference kernel in none. The harness and scope effects are 1.72× and 0.61× with Opus 4.8, and 1.37× and 0.60× with Fable 5.1. These ratios describe gaps between full configurations and do not isolate the contribution of any individual mechanism.

C.2 Per-Workload Latencies of the Definition-Level Comparison

Table 10 lists the per-workload latencies behind Table 9. All values were re-measured by the authors on one B200 with the official evaluator (`flashinfer-bench` main @ `40e6ca7`: CUPTI kernel span, cold L2, median, each definition's official tolerances) in at least two alternating rounds; the whole-definition solutions pass the official evaluator on every workload of their definitions, and only the three shapes used in the main text are timed here. The KDA-0.5 reproduction is the version at the end of the run. Library column: FA4 4.0.0b19 for FA4 backward; for DSA, the FlashInfer baseline solution of the contest dataset, which concatenates its inputs on every call and does not reflect the library kernel's own speed; for MoE, FlashInfer `trtllm_fp8_block_scale_moe`. The MoE Opus 5 whole-definition so-

Table 10: Per-workload device time behind Table 9 (μs, lower is better). Each cell is Opus 4.8 / Opus 5 / Fable 5.1.

Operator	Shape	JustGoal, whole definition	KDA-0.5 reproduction	Contest kernel	Library
FA4 backward	MHA	2675 / 1702 / 1932	1337 / 1416 / 1200	—	1371
	GQA	2701 / 1647 / 1565	1238 / 1478 / 1163	—	1456
	varlen MHA	3426 / 1985 / 2110	1557 / 1697 / 1378	—	1572
DSA forward	heavy load	19.3 / 9.58 / 13.3	14.1 / 8.30 / 7.43	10.13	585
	small request	10.0 / 4.02 / 4.73	5.93 / 4.41 / 3.56	3.36	661
	mixed lengths	13.5 / 6.90 / 9.30	9.22 / 6.29 / 5.11	6.92	652
GDN forward	long sequence	1126 / 213 / 178	710 / 126 / 234	124.9	—
	single short seq.	13.96 / 5.96 / 5.36	15.2 / 4.74 / 4.67	4.81	—
	few long seqs.	1863 / 198 / 203	973 / 138 / 312	107.7	—
MoE FP8	single token	294 / 300 / 53.7	135 / 72.9 / 42.1	52.0	149
	medium batch	551 / 433 / 431	336 / 386 / 280	275.8	435
	large prefill	2703 / 1390 / 2431	1271 / 1582 / 923	1028.5	2515

lution is bimodal across rounds on the single-token workload (63–122 μs); the median is shown. The sessions' own reports are generally more favorable than these numbers: the GDN Opus 4.8 session read a millisecond timer as microseconds, the MoE Opus 4.8 session's benchmark hit a CUDA graph cached on input pointers, and the MoE Opus 5 session timed itself on inputs it generated. These rows await the same-GPU paired re-measurement of Section 3.2.

Correctness was re-checked under the protocol of Section 3.2: the definition's reference implementation computed in fp32 as the oracle, three input seeds per workload, tolerance set relative to KDA-0.5's own error (FA4's for FA4 backward), judged separately for the maximum and the mean absolute error. The whole-definition solutions of all twelve operator-model combinations (fifteen directories, since FA4 backward has a fixed-length and a varlen definition) pass on every timed workload, with three exceptions: the Opus 4.8 and Opus 5 DSA solutions exceed the lse tolerance by 1.4–1.7$\times$ on the small-request workload, where KDA-0.5's lse is almost exact and the tolerance is tiny, both absolute errors being below 10^{-5}; and the Opus 4.8 MoE solution gets 3 rows wrong as a whole on large prefill, apparently near-ties in expert selection, with the mean error equal to KDA-0.5's, which the official evaluator passes through its 90% matched-ratio rule. In the KDA-0.5 reproduction, the Opus 4.8 DSA solution returns only the attention output and no lse, which the official evaluator did not catch; the Opus 4.8 MoE solution calls FlashInfer library kernels and fails the strict check above on two further workloads. Two items failed the public-repository retest as released: the three Opus 4.8 DSA KDA-0.5 reproduction implementations matched an earlier failing upstream version, and the Opus 5 MoE whole-definition solution hit a Triton launch-parameter mismatch on all 19 workloads (36 parameters expected, 27 given). Both pass after repair, and the tables use the repaired timing.